VITyarthi Project Report

Student Result & Grade Management System

Python Essentials 2

Student Name	ANYATAM WATS
Roll Number	26BCE11173
Faculty / Section	ANJALI MATHUR

Student Result & Grade Management System

1. Introduction

The Student Result & Grade Management System is a modular, menu-driven command-line application written in pure Python. It manages student records, subject-wise marks, automatic result calculations, class ranking and academic analytics. Records are persisted in JSON and can be exported to CSV.

The project demonstrates Python fundamentals, data structures, functions, modules, packages, object-oriented programming, validation, exception handling, file handling, logging and automated testing.

2. Problem Statement

Schools and colleges may maintain marks in scattered notebooks or spreadsheets. Manual calculation of totals, percentages, grades and ranks is time-consuming and can introduce calculation or data-entry errors. The proposed system centralizes student records and automates result processing while validating inputs.

3. Objectives

• Build a practical Python application for managing student academic results.

• Apply Python fundamentals including input/output, operators, control flow, functions and data structures.

• Demonstrate modular programming and object-oriented programming.

• Validate input and handle predictable errors safely.

• Persist records using JSON and provide CSV export.

• Generate useful report cards, rank lists and analytics.

• Demonstrate software quality through testing, logging and maintainable structure.

4. Functional Requirements

ID	Requirement	Implementation
FR-01	Add student	Validated roll number, name and course; duplicate protection.
FR-02	View students	Structured student table.
FR-03	Update student	Validated name/course update.
FR-04	Delete student	Confirmation before deletion.
FR-05	Search	Case-insensitive roll/name/course search.
FR-06	Marks	Subject-wise marks, 0–100 validation.
FR-07	Remove marks	Delete a subject mark.
FR-08	Result calculation	Total, percentage, grade, GPA, PASS/FAIL.
FR-09	Ranking	Percentage ranking with ties.
FR-10	Reports	Report card, rank list, analytics.
FR-11	Export	CSV export.
FR-12	Persistence	JSON load/save with atomic replacement.

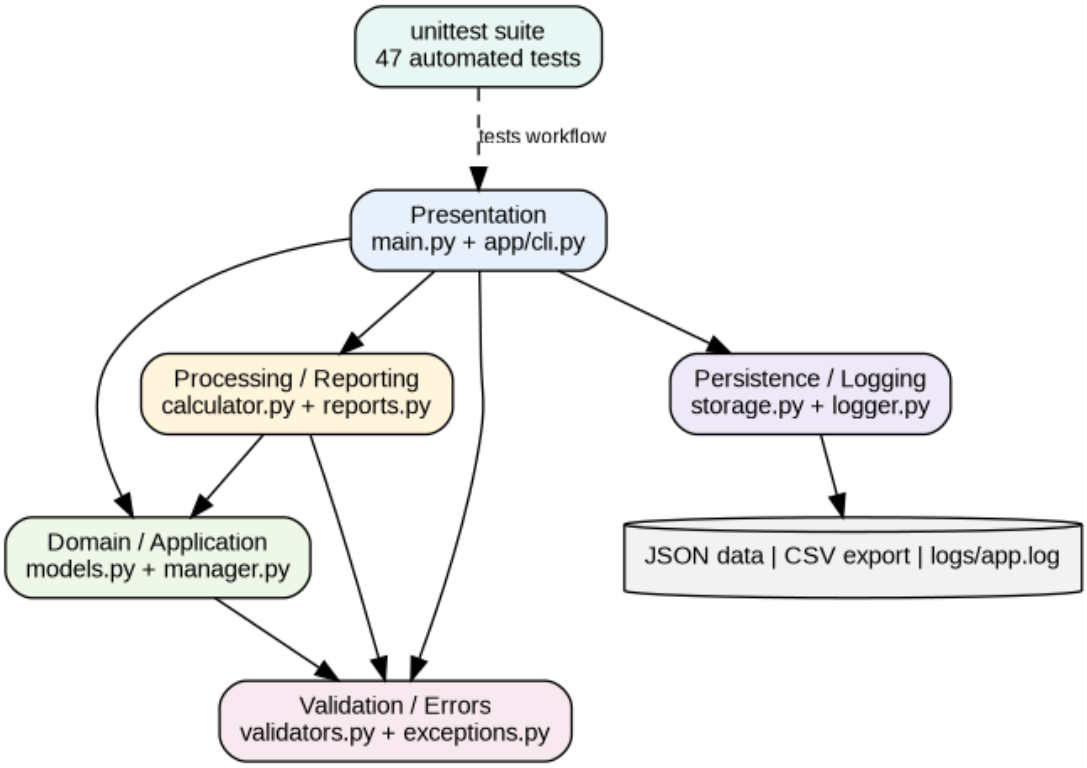
5. Non-Functional Requirements

ID	Requirement	Evidence
NFR-01	Usability	Menu-driven CLI and clear prompts.
NFR-02	Reliability	Atomic saves and controlled corrupt-file errors.

ID	Requirement	Evidence
NFR-03	Maintainability	Separated modules by responsibility.
NFR-04	Error handling	Custom exceptions and validation loop.
NFR-05	Performance	Dictionary keyed by roll number for average O(1) lookup.
NFR-06	Resource efficiency	Python standard library and lightweight JSON storage.
NFR-07	Observability	Application/storage logging.
NFR-08	Testability	Injected CLI input/output and 47 tests.

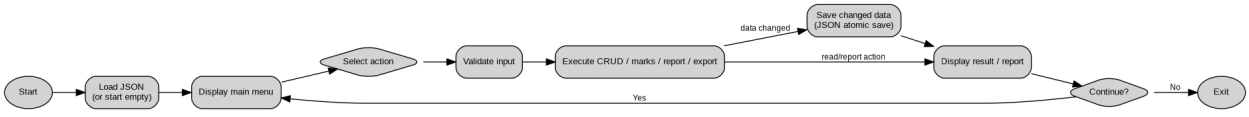
6. System Architecture

The application is organized into presentation, domain/application, processing/reporting, validation/error and persistence/observability responsibilities. This keeps the CLI separate from calculations and storage and makes individual components easier to test.



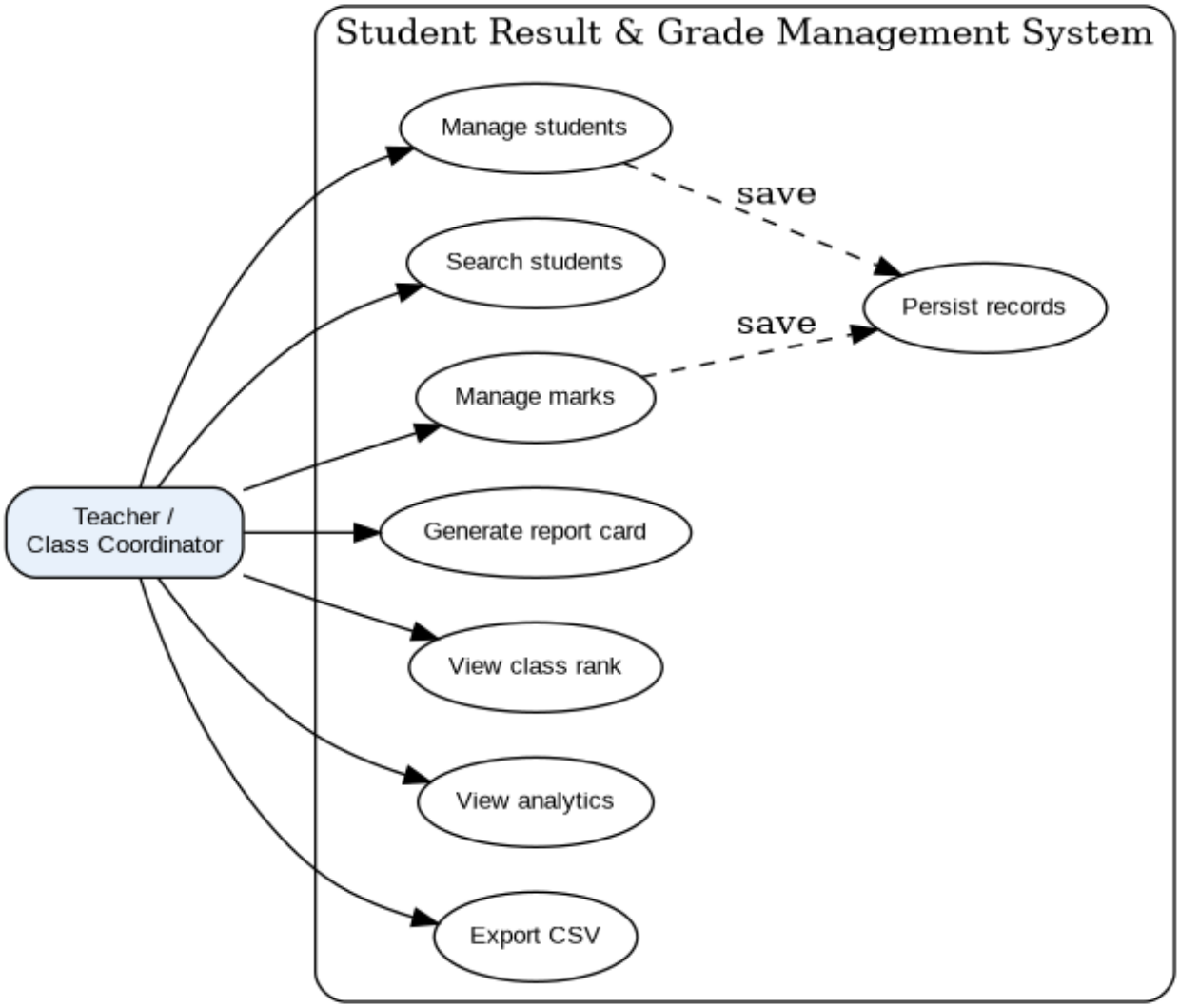
7. Process Flow / Workflow

At startup the configured JSON file is loaded. The user selects a menu action; inputs are validated; the selected operation is executed; changed data is persisted; and the resulting output is displayed. The cycle continues until the user exits.



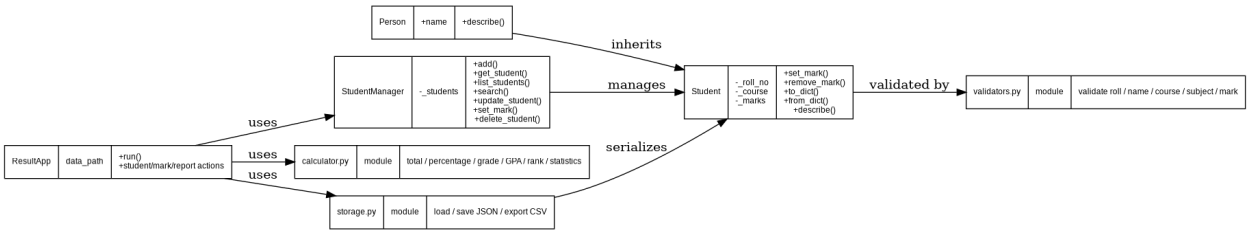
8. Use Case Diagram

The main actor is the teacher/class coordinator. Department office staff can perform the same operational tasks. The major use cases are student management, search, marks management, reporting, ranking, analytics, export and persistence.



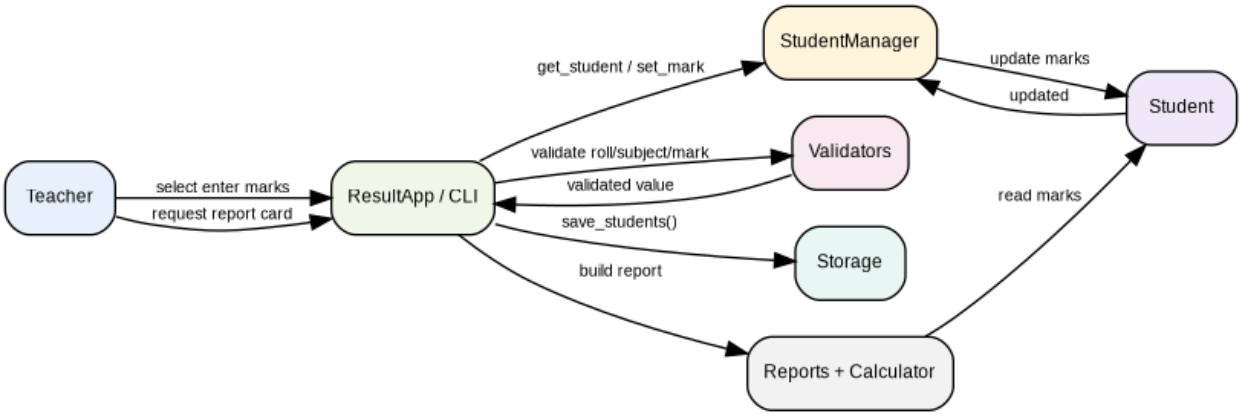
9. Class / Component Diagram

The core domain model contains Person and Student. StudentManager owns the student collection and exposes CRUD/marks operations. ResultApp coordinates the workflow and uses calculator, storage and validation modules.



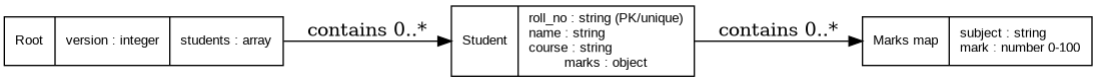
10. Sequence Diagram

The sequence below describes entering a mark and later generating a report card. Validation occurs before the mark reaches the Student object, and successful changes are persisted.



11. Database / Storage Design

The project does not use a relational database. It uses a JSON document whose root contains a version and a list of student records. Each student has a unique roll number and a subject-to-mark mapping. CSV is used as an export format.



Integrity rules: roll number uniqueness; validated text fields; marks in the inclusive range 0–100; pass requires every recorded subject to meet the 40-mark pass threshold; atomic save through a temporary file and os.replace().

12. Design Decisions & Rationale

• **Standard library:** keeps installation simple and demonstrates Python directly.

• **JSON storage:** fits the hierarchical student → subjects → marks structure without a database server.

• **Dictionary by roll number:** supports fast lookup and duplicate detection.

• **Separate calculator module:** makes result logic reusable and independently testable.

• **Custom exceptions:** distinguish validation, missing-record and storage failures.

• **Atomic saves:** reduce the risk of partial JSON files.

• **Dependency injection:** makes the interactive CLI testable using scripted input/output.

13. Implementation Details

Module	Responsibility
main.py	Application entry point and command-line argument parsing.
cli.py	Menu, prompts, workflow and user-facing output.
models.py	Person/Student domain classes, validation hooks and serialization.
manager.py	CRUD, search and mark management.
calculator.py	Totals, percentage, grades, GPA, ranking and statistics.
reports.py	Formatted report card, ranking and analytics output.
validators.py	Input validation and normalization.
storage.py	JSON load/save and CSV export.
logger.py	Logging configuration.

Module	Responsibility
exceptions.py	Custom exception hierarchy.

The project also demonstrates lists, dictionaries, sets, tuples, arrays, functions, control flow, file handling, inheritance, encapsulation and polymorphism.

14. Screenshots / Results

The following screenshots were generated from the project using its sample dataset. They can also be replaced by your own terminal screenshots after running the application locally.

Main Menu

```
STUDENT RESULT & GRADE MANAGEMENT SYSTEM
================================================================
1. Add a student
2. View all students
3. Update a student
4. Delete a student
5. Search students
6. Enter / update marks
7. Remove a subject mark
8. Student report card
9. Class rank list
10. Analytics dashboard
11. Export results to CSV
0. Exit
```

Student List

```
Roll No   Name                Course          Subj       %  Gr Result
-----------------------------------------------------------------------
CS101     Asha Rao            B.Tech CSE         4   85.75  A+ PASS
CS102     Bala Iyer           B.Tech CSE         4   68.00  B  PASS
CS103     Chitra Nair         B.Tech CSE         4   92.25  O  PASS
CS104     Deepak Verma        B.Tech CSE         4   42.00  D  FAIL
CS105     Esha Kapoor         B.Tech CSE         4   80.00  A+ PASS
CS106     Farhan Sheikh       B.Tech CSE         4   65.50  B  PASS
CS107     Gita Menon          B.Tech CSE         4   56.25  C  PASS
CS108     Harish Patel        B.Tech CSE         0    0.00  -  NO MARKS
```

Individual Report Card

```
======================================================================
                        STUDENT REPORT CARD
======================================================================
Roll No : CS103
Name    : Chitra Nair
Course  : B.Tech CSE
----------------------------------------------------------------------
Subject                          Marks   Grade  Points
----------------------------------------------------------------------
English                          94.00       O      10
Mathematics                      91.00       O      10
Physics                          89.00      A+       9
Python                           95.00       O      10
----------------------------------------------------------------------
Total       : 369 / 400
Percentage  : 92.25%
Grade       : O
GPA         : 9.75
Result      : PASS
```

Class Rank List

```
======================================================================
                          CLASS RANK LIST
======================================================================
Rank  Roll No   Name                      Total        %  Gr Result
----------------------------------------------------------------------
1     CS103     Chitra Nair                 369    92.25  O  PASS
2     CS101     Asha Rao                    343    85.75  A+ PASS
3     CS105     Esha Kapoor                 320    80.00  A+ PASS
4     CS102     Bala Iyer                   272    68.00  B  PASS
5     CS106     Farhan Sheikh               262    65.50  B  PASS
6     CS107     Gita Menon                  225    56.25  C  PASS
7     CS104     Deepak Verma                168    42.00  D  FAIL
----------------------------------------------------------------------
1 student(s) without marks are not ranked.
```

Analytics Dashboard

```
======================================================================
                          CLASS SUMMARY
======================================================================
Total students   : 8
With marks       : 7
Passed / Failed  : 6 / 1
Pass rate        : 85.7%
Class average    : 69.96%
Highest / Lowest : 92.25% / 42.00%
Std deviation    : 16.21
Top performers   : Chitra Nair (92.25%), Asha Rao (85.75%), Esha Kapoor (80.00%)

======================================================================
                      SUBJECT-WISE ANALYSIS
======================================================================
Subject                    N     Avg    High     Low  StdDev   Pass%
----------------------------------------------------------------------
English                    7   72.43      94      52   14.21   100.0
Mathematics                7   69.57      91      45   15.98   100.0
Physics                    7   65.86      89      33   17.83    85.7
Python                     7   72.00      95      38   18.52    85.7

======================================================================
                        GRADE DISTRIBUTION
======================================================================
O  | # 1
A+ | ## 2
A  |  0
B  | ## 2
C  | # 1
D  | # 1
F  |  0
```

15. Testing Approach

The project uses Python unittest. Tests cover validation, models, calculations, storage, manager CRUD/search/marks operations and scripted CLI workflows.

Verification command: python -m unittest discover -v

Observed result: Ran 47 tests in 0.019s — OK. All 47 tests passed during preparation of this report.

16. Challenges Faced

• Separating interaction, business logic and persistence while keeping the project understandable.

• Preventing invalid marks and malformed records.

• Handling duplicate roll numbers and missing students consistently.

• Making JSON saves safer against interrupted writes.

• Handling ties and students with no marks in ranking.

• Making an interactive CLI testable without manual input.

• Computing subject statistics when students may have different subject sets.

Solutions Applied

• Dedicated validators and custom exceptions.

• StudentManager as the central CRUD/domain service.

• Atomic JSON save using a temporary file and os.replace().

• Pure calculation functions reused by reports and export.

• Injected input/output functions for CLI tests.

• Explicit numeric statistics over the available subject marks.

17. Learnings & Key Takeaways

• How to divide a Python application into cohesive modules.

• How inheritance, encapsulation and polymorphism can model domain objects.

• How different Python data structures support different tasks.

• How validation and custom exceptions improve reliability.

• How JSON and CSV can be implemented with the standard library.

• How automated tests verify both functions and menu workflows.

• How logging supports troubleshooting.

• How diagrams and requirements communicate a software solution beyond code.

18. Future Enhancements

• Graphical desktop or web interface.

• Authentication and role-based access.

• Database-backed storage for larger deployments.

• PDF report-card generation.

• Weighted subjects and credit-based GPA.

• Backup/restore and CSV import.

• Configurable grading scales.

• Richer visual analytics dashboard.

19. References

1. VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines, course-provided project brief.

2. Python 3 Documentation — standard library documentation for json, csv, logging, unittest, argparse, array, math and file handling.

3. Git Documentation — version-control concepts and command reference.

4. Project source code, tests, README.md, statement.md and sample data.

20. VITyarthi Compliance Checklist

Requirement	Status
3+ functional modules	Covered
Clear input/output and workflow	Covered
4+ non-functional requirements	Covered and documented
5–10 meaningful modules/files	Covered (10 app modules)
Validation and error handling	Covered
Testing	Covered — 47 tests passed
README.md	Covered
statement.md	Covered
Problem statement/objectives	Covered
Functional/non-functional requirements	Covered
Architecture diagram	Included
Workflow diagram	Included
Use Case diagram	Included
Class diagram	Included
Sequence diagram	Included
Storage design	Included
Design decisions/rationale	Included
Implementation details	Included
Screenshots/results	Included
Testing approach	Included
Challenges	Included
Learnings	Included
Future enhancements	Included
References	Included
Project report PDF	This document

GitHub note: this ZIP can contain the complete source and documentation, but actual Git commit history cannot be demonstrated by a ZIP alone. Before submission, create/push the repository and make meaningful commits. Replace the clone placeholder in README.md with the actual repository URL.